
Introduction to RDF & SPARQL

Welcome

Meet the trainer:

Joe Fagan
Knowledge Graph Consultant
joe.fagan@stardog.com
+44 7788 148772



Course Overview

In this course, students will learn:

- to write SPARQL queries
- to use SPARQL queries to explore and edit an RDF graph

Audience:

- The course is designed for students who wish to become knowledge graph engineers

Prerequisites:

- An understanding of RDF triples, IRIs and Namespaces from the Advanced Designer Course:
<https://academy.stardog.com/learn/courses/23/advanced-designer/lessons/183/adv-designer-iris-triples-and-namespaces>



Sparql Query Language Overview

- SPARQL is a query language specifically designed for querying data represented in the Resource Description Framework (RDF) format.
- Sparql Query Language allows users to extract specific information from large datasets by querying based on patterns, conditions, and relationships.
- This course will show how Sparql Query Language may be used to explore any RDF dataset.
 - The queries will become progressively more complex.
 - The examples are shown using the Stardog Platform, but can be run on any RDF database or Knowledge Graph Platform



Sparql Query Language Agenda

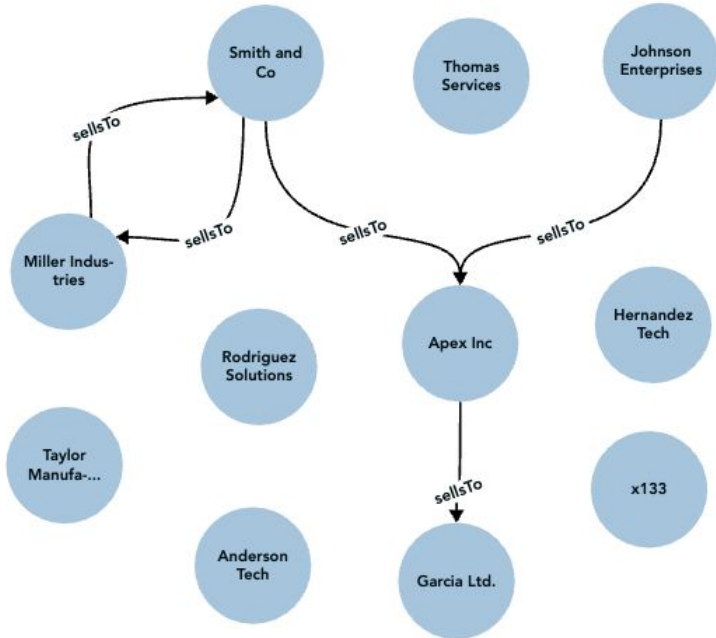
We will work many examples queries from very simple to more complex

- Basic Triple Patterns
- Basic Graph Patterns (BGP): Combination of triple patterns.
- FILTERs: These restrict values returned based on conditions.
- OPTIONAL Patterns: Match patterns if possible, but return results even if there's no match.
- UNION and MINUS Patterns: Return results that match either of the given patterns or subtract results from a set
- Aggregations (GROUP BY, COUNT, SUM, etc.)
- Subqueries: SPARQL queries embedded within other queries

This course follows the SPARQL 1.1 latest specification at <https://www.w3.org/TR/sparql11-query/>



The dataset – a visual representation



We have a simple dataset of our 11 customers.

Some customers sell to other customers

For each customer, we have additional attributes, like employee count and telephone number

Smith and Co

customer

●→ employee_count	1000
●→ hasTelephone	405-123-1124



Looking at some triples

Subject	Predicate	Object
:x123	a	ex:Customer
:x123	rdfs:label	Smith and Co
:x123	rdf:description	Great customer!
:x123	ex:tel	405-123-1124
:x123	ex:employees	1000
:x123	ex:sellsTo	:x125
:x123	ex:sellsTo	:x126
:x124	a	ex:Customer
:x124	rdfs:label	Johnson Enterprises
:x124	ex:tel	405-123-1125
:x124	ex:employees	950
:x124	ex:sellsTo	:x126

Here are some sample triples from the dataset.

- Our prefixed IRIs are shown in **Red**
- Industry standard prefixed IRIs are in **Blue**
- Literal values are shown in **Green**

An RDF graph consists entirely of triples, sometimes millions or billions!



Basic Triple Patterns

The SELECT clause is the most common query.

Ex 1: Return a table showing every triple in the default graph

```
select ?s ?p ?o
where
{
    ?s ?p ?o
}
order by ?s
limit 100
```

Tip: To run the query in Stardog Studio, place the cursor anywhere in the query and click Run or Cmd-Enter. Do not highlight the query.

Notice similarity to SQL!

- “Where” is optional
- “Order By” is optional - specifies sorting of results
- “Limit” is optional - specifies the number of results
- Extra Whitespace is ignored!
- **Select *** means show all variables in the pattern

Ex 2:

```
select * {?a ?b ?c}
```

Gives similar but unsorted results and labels the column as a, b, c.



Basic Triple Patterns – One variable examples

Can have 3, 2, 1 or 0 variables in each triple in the pattern, inside the curly brackets.

EG Fix the S and P and find triples that match that pattern and return results in the variable **?xyz**

```
select * { :x123 a ?xyz }
```

Here we Fix the Subject and Object
Find the IRI that links :x123 to their tele #

```
select * { :x123 ?p "405-123-1124" }
```

Fix the Predicate and Object
Find the IRI of the named customer

```
select * { ?who rdfs:label "Smith and Co" }
```

Notice variables must begin with ? or \$

\$abc is the same variable as ?abc

Variables are case sensitive ?xy != ?Xy

Variables can have any of (a-z, A-Z, 0-9, _) and many accented characters like Õ, Ç

Valid: ?name_1st ?x_age \$totalCount ?ÕÊÇ

Invalid: ?name-first ?x.age ?sum/num

Tip: The only allowed punctuation mark in a variable name is underscore _



Basic Triple Patterns – Two variable examples

Find all IRIs that are instances of a class

```
Select ?IRI { ?IRI a ?class }
```

Find every label

```
SELECT ?label { ?iri rdfs:label ?label }
```

Find all the customers who sell to other customers and rename the column?

```
select (?cust1 as ?Seller)  
{ ?cust1 ex:sellsTo ?cust2 }
```

Find all triples with :x123 as the subject

```
select * { :x123 ?p ?o }
```

All keywords are case-insensitive - with the exception of 'a' which is shorthand for 'rdf:type'

So you can write Select or SELECT or eg SeLeCT

The list of variables after Select are the names of the variables that will be returned in the results set

You can rename a column of output with

Select (?old_varName AS ?new_Name) ...



Adding new variables in the Select Clause

We saw you can change a column label to a new variable name

```
select (?lab AS ?Customer_Name)  
{ :x123 rdfs:label ?lab }
```

Rename multiple columns. Change order

```
select (?s as ?Subject) (?o as ?Object) (?p  
as ?Predicate)  
{?s ?p ?o}
```

New vars do not need to depend on data

```
select (1+1 as ?two) {}  
select (1+1 as ?two) (?two * 3 as ?six) {}
```

New variables can be used just to rename columns or introduce new values

New variables do not need to depend on the data in the graph. We can assign (or bind) values to variables in the Select clause. Notice we still need a Where clause, ie {}, even if it is empty.

New variables can also be re-used later in expressions as we did with `?two`

This gives us a powerful way to manipulate results



Basic Graph Patterns #1

Find the iri with label "Smith and Co". Then find the telephone number from that iri

```
select ?telephone {  
  ?s rdfs:label "Smith and Co" .  
  ?s ex:tel ?telephone .  
}
```

Find who sells to whom

```
select (?xLabel as ?Seller) ?yLabel {  
  ?x a ex:Customer .  
  ?y a ex:Customer .  
  ?x ex:sellsTo ?y .  
  ?x rdfs:label ?xLabel .  
  ?y rdfs:label ?yLabel .  
}
```

Usually we want to further qualify the results to be matched - a single triple pattern is not enough.

Then a Basic Graph Pattern is used IE a set of triple patterns that gets matched simultaneously.

Note: order doesn't matter. We could have written

```
select ?telephone {  
  ?s ex:tel ?telephone .  
  ?s rdfs:label "Smith and Co" .  
}
```



Basic Graph Patterns #2

Name and tel of cust with 950 employees

```
select ?sLabel ?telephone {  
  ?s a ex:Customer .  
  ?s ex:employees 950 .  
  ?s ex:tel ?telephone .  
  ?s rdfs:label ?sLabel .  
}
```

Or using Turtle shorthand

```
select ?sLabel ?telephone {  
  ?s a ex:Customer ;  
    ex:tel ?telephone ;  
    ex:employees 950 ;  
    rdfs:label ?sLabel .  
}
```

Fully written out triples (or a chain of triples) end with a .

But we can use ; when the subject is the same as the previous triple

And we can use , when subject and predicate are the same

Triples in the BGP must be connected through shared variables. Notice the variable **?s** appears in all triples.

Let's look at cases where we break this rule



Section End Quiz

Consider the query shown.

1. How many columns in the output?
2. What are the column headers and what does each column contain?
3. If I wrote instead `'select *'` how many columns would there be in the output?
4. How could I rename the `'sLabel'` column to `'Name'`?
5. How could I sort the results on `'sLabel'`?
6. What happens if I change `?s` to `$s` in one triple?
7. How could I limit the number of results?
8. What changes if I change the order of the triples in the query?

Build a telephone list

```
select ?sLabel ?telephone
{
  ?s a ex:Customer .
  ?s ex:tel ?telephone .
  ?s rdfs:label ?sLabel .
}
```



The Cartesian Product problem

Here is an example that will produce a cross product

```
select *  
{  
  ?cust1 a ex:Customer .  
  ?cust2 a ex:Customer .  
}
```

Notice the variable named ?s0 below

```
select ?sLabel ?telephone {  
  ?s0 a ex:Customer .  
  ?s0 ex:employees 950 .  
  ?s0 ex:tel ?telephone .  
  ?s0 rdfs:label ?sLabel .  
}
```

If the triples are not connected to each other you may end up with unexpected results - the cross product of actually 2 (or more) result sets.

There must be no isolated triples!!

The first example will bind all 11 customers to ?cust1 and also ?cust2 giving all combinations of ?cust1 and ?cust2 and so 121 results.

The second will bind ?s0 to the customer with 950 employees and cross product that with ?s0 which is bound to every IRI that looks like ?s0 ex:tel ?telephone .

The join condition in Sparql is not written explicitly; it is implied by the variables that patterns share. If they share no variables, the join condition is vacuously true and the result is a Cartesian product.



The Cartesian Product problem #2

```
{  
  ?person a foaf:Person .  
  ?person foaf:name ?personName .  
  ?person foaf:knows ?friend .  
  ?friend foaf:name ?friendName .  
  
  ?work ex:title ?title .  
  ?work ex:hasManager ?manager .  
  ?manager foaf:name ?managerName .  
  
  ?person ex:worksAt ?work .  
}
```

To avoid the problem, you should be able to 'pull a thread' between variables so that every triple is 'on the thread'

The first 4 triples look good together, connected with **?person** and **?friend**

The next 3 look good together via **?work** and **?manager**

But the patterns look disconnected!!! However the last triple, via **?person** and **?work** connects the 'islands'

Notice: To check BGP, you don't need to understand the query!
A more readable order would make this check more easy. IE
move the last triple up to avoid the surprise!



The Cartesian Product problem #3

```
WHERE {  
  ?person a foaf:Person .  
  ?person foaf:name ?personName .  
  ?person foaf:knows ?friend .  
  ?friend foaf:name ?friendName .  
  
  ?work ex:title ?title .  
  ?work ex:hasManager ?manager .  
  ?manager foaf:name ?managerName .  
  
  ?person ex:worksAt ?work .  
}
```

The lines show how the triples are connected by their variables.

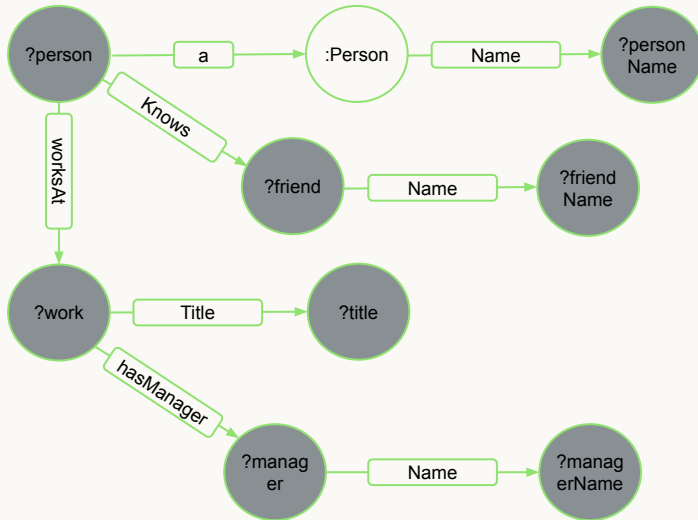
A Basic Graph Pattern should consist of just one pattern.

If we had omitted the last triple we would have the cartesian or cross product between the results sets of the first four and next three patterns.

(Here we find a person who has a name , knows a friend who has a name. The person works somewhere and has a title, and has a manager who has a name.)



The BGP must be connected



WHERE {

`?person a foaf:Person .`

`?person foaf:name ?personName .`

`?person foaf:knows ?friend .`

`?friend foaf:name ?friendName .`

`?work ex:title ?title .`

`?work ex:hasManager ?manager .`

`?manager foaf:name ?managerName .`

`?person ex:worksAt ?work .`

}



Math functions and operators

There is a full suite of mathematical operators and functions available

`select`

`(pi() as ?pi)`

`(pow(?pi, 4) as ?pi4)`

`(sqrt(sqrt(?pi4)) as ?pi1)`

`(round(?pi * 1000) / 1000 as ?pi_rounded)`

`(ceil(?pi) as ?ceil)`

`(floor(?pi) as ?floor)`

`(sin(2*pi()) as ?sin_pi )`

`}`

Note: The Sparql spec includes the usual operators `+`, `-`, `/`, `*`

And `ceil()`, `floor()`, `abs()`, `round()` and `rand()`

But almost every Sparql implementation will give you `pi()`, trig and log functions, and lots more.



String functions

```
select
("Hello World" as ?myString)
(lcase(?myString) as ?lcase)
(ucase(?myString) as ?ucase)
(strstarts(?myString, "Hell") as ?strtstarts)
(strends(?myString, "Hello") as ?strends)
(contains(?myString, "o Wor") as ?contains)
(strbefore(?myString, "World") as
?strbefore)
(strafter(?myString, "l") as ?strafter)
{}
```

Sparql has a rich collection of string functions some of which are shown here. They behave intuitively according to the function name.

In addition there are date functions, HASH functions, language functions and IRI functions.

We see that we can assign a value to a variable in the select clause.

Eg ("Hello World" as ?myString) {}

But this new variable **?myString** is not available in the where clause. To assign values inside the where clause we can use the bind() function - which we will look at next.



The Bind function

```
# bind "sparql" to the variable ?result
select ?result
{ bind("sparql" as ?result) }
```

```
# use bindings in the triple pattern
select *
{
  bind (:x123 as ?s)
  bind (ex:employees as ?p)
  ?s ?p $employees .
  bind ( $employees * 0.10 as $execs)
}
```

With the bind() function we assign a value to a variable inside the where clause. We can use the binding in our triple pattern

Unlike triple patterns themselves,, the order of bind() statements matter. They are executed sequentially.

A great way to assign multiple values is the Values statement, which we will look at next.



The Values clause

get me the iri's of 3 specific companies

```
select ?co_iri {  
  VALUES ?co_name {  
    "Smith and Co"  
    "Johnson Enterprises"  
    "Miller Industries"  
  }  
  ?co_iri rdfs:label ?co_name  
}
```

find the bigger value of each pair of numbers

```
select ?row_max  
{  
  VALUES (?x ?y) { (1 3) (4 2) (3 7) }  
  bind(IF(?x > ?y, ?x, ?y) AS ?row_max)  
}
```

The VALUES clause provides inline data for variable.

Commonly used for filtering results and for debugging.

Values clause supports both single and multiple variable bindings. In the example the tuple (?x ?y) is assigned values.

Notice the new IF clause to implement our own ?max. Notice the inbuilt MAX() compares values in one column, not across rows.



The Values clause

Find customer location from telephone area code

```
select ?tel ?area_code ?location
{
  ?x ex:tel ?tel .
  bind(strbefore(?tel, "-") as ?area_code)
  values (?area_code ?location)
  {
    ( "405" "Oklahoma City")
    ( "303" "Denver metroplitan area")
    ( "315" "Northern New York")
    ( "252" "Northeastern North Carolina")
  }
}
```

The VALUES clause can be used as a simple lookup table

Here we're grabbing the area code from the telephone number and using that to find the location

Notice BIND must come before VALUES



The IF clause

Categorise based on the number of employees

```
select *  
{  
  ?x a ex:Customer ;  
    rdfs:label ?name ;  
    ex:employees ?emp .  
  bind(  
    IF($emp > 500, "Large employer",  
    "Small employer") as ?type )  
} order by ?type
```

The IF clause enables conditional logic.

IF(condition, trueValue, falseValue)

- condition: The expression to evaluate. It must return a boolean value (true or false).
- trueValue: The value to return if the condition is true.
- falseValue: The value to return if the condition is false.

IF (?x=1, "x is 1", "x is not 1)



The Filter clause

Find companies with between 500 and 1000 employees

```
select ?xLabel {  
  ?x a ex:Customer ;  
  ex:employees $emp ;  
  rdfs:label ?xLabel .  
  FILTER ( $emp > 500 && $emp < 1000 )  
}
```

Find companies with name starting with 'Sm' or ending with 'Tech'

```
select * {  
  ?x a ex:Customer .  
  ?x rdfs:label ?name .  
  FILTER ( strStarts(lcase(?name), "sm")  
    || strEnds(?name, "Tech" ) )  
}
```

The FILTER clause restrict results based on conditions.

Arithmetic: *, /, +, -

Comparison: =, !=, <, >, <=, >=

Logical: ! (NOT), && (AND), || (OR)

Results that satisfy the FILTER make it

FILTER can also use any other available function.

With FILTER you can create more precise and targeted Sparql queries.

Filter keeps what filters as true

Adding a Filter will never increase the number of results! To add more results you can use Optional clause or Union



OPTIONAL clause

#q1 give me the IRIs of all customers, and their labels - if they have a label

```
select * {  
  ?x a ex:Customer .  
  optional { ?x rdfs:label ?name . }  
}
```

#q2 Create a table of customers, incl. blanks.

```
select * { ?x a ex:Customer .  
  optional { ?x rdfs:label ?name }  
  optional { ?x rdf:description ?desc }  
  optional { ?x ex:employees $emps }  
  optional { ?x ex:sellsTo ?y }  
  optional { ?x ex:tel ?tel }  
} order by ?x ?name
```

Without OPTIONAL the pattern is mandatory, and only data that matches ALL criteria in the pattern is returned. With OPTIONAL, you can still retrieve partial data even if the optional criteria are not met.

EG q1 If we use this query, customers without label would be omitted

```
select * {  
  ?x a ex:Customer ; rdfs:label ?name .  
}
```

Without OPTIONAL in q2 only rows in the resulting table without blanks would be included



UNION keyword

Retrieve the company name, whether the predicate is `rdfs:label` or the custom predicate `ex:Name`

```
select * {  
  ?x a ex:Customer .  
  { ?x rdfs:label ?string }  
  UNION  
  { ?x ex:Name ?string }  
}
```

Use property paths

```
SELECT ?x ?name {  
  ?x a ex:Customer .  
  ?x ( rdfs:label | ex:Name ) ?name .  
}
```

The UNION keyword is used to combine the results of two or more patterns

UNION is used often when data is stored in different formats, to combine results from multiple patterns, regardless of how it is represented in the dataset

UNION has performance trade-offs. Limit the dataset by using a pattern outside the UNIONS

Simple UNION can often be replaced with property paths. The pipe symbol means match either `rdfs:label` **OR** `ex:Name`

We will look at property paths next



DISTINCT and REDUCED Keywords

Find all the predicates in my data

Remove duplicate rows

```
select DISTINCT ?pred
```

```
{
```

```
  ?x ?pred ?y
```

```
}
```

REDUCED will almost always give same results but is much faster on larger datasets

```
select REDUCED ?seller
```

```
{
```

```
  ?seller ex:sellsTo ?buyer
```

```
}
```

DISTINCT removes duplicate rows in the output. It is a modifier (not a function) and brackets are not used.

Often, when not all variables in the query are projected you will get duplicate results.

TIP: DISTINCT removes all duplicates exactly and so can be resource-intensive for large result sets. In contrast, REDUCED partially removes duplicates quickly but some duplicates may still remain.



FILTER NOT EXISTS filter expression

Find customers who do not have a label

```
select * {  
  ?x a ex:Customer .  
  filter not exists { ?x rdfs:label ?xLabel }  
}
```

Find companies missing label or employee count

```
select * {  
  ?x a ex:Customer .  
  filter not exists {  
    ?x rdfs:label ?y ;  
    ex:employees $num_employees  
  }  
} order by ?x
```

Sparql negation

FILTER NOT EXISTS takes a graph pattern (as opposed to just criteria on the bound variables like FILTER alone). It goes back to the graph

With FILTER NOT EXISTS you can create very precise and targeted Sparql queries.

FILTER NOT EXISTS behave very similar to MINUS, which we will look at next.



MINUS clause

Find customers who do not have a label

```
select * {  
  ?x a ex:Customer .  
  MINUS { ?x rdfs:label ?xLabel }  
}
```

Exclude customers who sell to each other

```
select * {  
  ?x a ex:Customer .  
  MINUS { ?x ex:sellsTo ?y . ?y ex:sellsTo ?x }  
} order by ?x
```

The MINUS clause removes any solutions that match the pattern inside it from the results of the main query.

If the pattern inside MINUS is found for a given solution, that solution is excluded from the results.

MINUS and FILTER NOT EXISTS can have different performance profiles.

Results may differ in obtuse circumstances!!!
MINUS { pattern to exclude } versus
FILTER NOT EXIST { pattern to check for non existence }



Anonymous node syntax

We can use anonymous node syntax to make the query more readable

```
select distinct ?pred  
{ [] ?pred [] }
```

Find customers who mutually sell to each other

```
select *  
{  
  _:seller ex:sellsTo _:buyer .  
  _:buyer ex:sellsTo _:seller .  
  _:seller rdfs:label ?mutual_customers  
}
```

If a variable in the subject or object position is not needed in the output, you can refer to the resource as []

It means I don't care what the value is.

If you need to reuse the subject or object resource within the pattern, you can assign it a name, such as _:x



Aggregates COUNT, SUM, AVG, MIN, MAX

Aggregates on numbers

```
select ( COUNT (?emps) as $Count )  
( MAX (?emps) as $Max )  
( MIN (?emps) as $Min )  
( SUM (?emps) as $Sum )  
( AVG (?emps) as $Avg )  
( $Sum/$Count as $MyAvg ) # variable reuse  
{  
  ?x a ex:Customer ; ex:employees ?emps  
}
```

gather company names into a csv string

```
select ( GROUP_CONCAT (?name ; separator  
= ", ") as ?csv_string )  
{ ?x a ex:Customer ; rdfs:label ?name }
```

Aggregates allows us to summarise and analyze RDF data. For numbers, we can COUNT, SUM, AVG, MIN, MAX, GROUP_CONCAT

For non-numbers, like dates, strings or iri's, we can only COUNT or GROUP_CONCAT.



Aggregates with GROUP BY clause

How many tel nums in each area code?

```
select ?areaCode ( count(?x) as $count )  
{ ?x a ex:Customer ; ex:tel ?phone .  
  BIND(SUBSTR(?phone, 1, 3) AS ?areaCode)  
} GROUP BY ?areaCode
```

Average num employees by areacode?

```
select ?areaCode (avg(?cust) as $avg)  
(sum(?emps) as $numEmps)  
{  
  ?cust a ex:Customer ; ex:tel ?phone .  
  BIND(SUBSTR(?phone, 1, 3) AS ?areaCode)  
  ?cust ex:employees ?emps  
} GROUP BY ?areaCode
```

The GROUP BY clause is used to group query results based on one or more variables.

Useful to perform aggregate operations on grouped data, such as counting the number of items in each group, calculating averages, or finding minimum and maximum values.

Aggregates, with GROUP BY is conceptually similar to pivot tables in spreadsheet.

A good practice is to run your query first without GROUP BY to clearly understand the data. Use as few variables as possible, with descriptive names



Sparql Subqueries

The inner query we want to find the aver is.

```
select (avg(?num) as ?avg)  
{ ?x ex:employees ?num }
```

We then build the whole query

```
select *  
{  
  ?x ex:employees ?num1  
  filter (?num1 > ?avg)  
  {  
    select (avg(?num) as ?avg)  
    { ?x ex:employees ?num }  
  }  
}
```

We use sub-queries

- Either for clarity - make query more modular
- Or when we must go back to the graph a second time with values from the first pass

When it is impossible to satisfy the query in one pass we will need a sub-query



Property Paths

A common use of a Sequence Path is when you find an iri but really want only its label

```
select ?bLabel
```

```
{ :x123 ex:sellsTo / rdfs:label ?bLabel }
```

As shortcut for

```
{ :x123 ex:sellsTo ?y . ?y rdfs:label ?bLabel }
```

Return all subject and objects of all triples except those with the given predicates

```
select * { ?s !( ex:tel | a | ex:employees ) ?o }
```

Find every possible path or any length along sellsTo in reverse

```
select * { ?seller ^ex:sellsTo+ ?buyer }
```

Property paths offer flexibility in managing variable length paths and in writing more expressive and efficient queries.

Here is a list ordered by usefulness

Syntax

$\wedge p$

p_1 / p_2

$p_1 \mid p_2$

(p)

p^+

p^*

$p?$

Matches

Inverse path (object to subject).

A sequence path: p_1 followed by p_2 .

A alternative path: p_1 or p_2

Use $()$ to control precedence

One or more matches of p

Zero or more matches of p

Zero or one match of p



Sparql Query Forms

The SELECT query form is just one of four query forms in the Sparql Query Language

- SELECT query form is the most common. It extracts data from the RDF graph and returns variable bindings in a tabular form.

The other three query forms, that we will look at, are

- ASK query form is used to determine whether a query pattern matches or not. It returns a boolean value (true or false).
- DESCRIBE query form is used to retrieve an RDF graph that describes the resources found. The exact content of the description is left to the SPARQL endpoint creator. It's not specified in the spec.
- CONSTRUCT is used to extract data and transform the results into RDF triples. It returns a new RDF graph based on the query pattern.



CONSTRUCT query form

Return the whole graph as RDF

```
CONSTRUCT { ?s ?p ?o } # template  
where { ?s ?p ?o }      # pattern
```

Create RDF list of customer names using
ex:Name instead of rdfs:label

```
CONSTRUCT { ?cust ex:Name ?name }  
{  
    ?cust a ex:Customer .  
    ?cust rdfs:label ?name .  
}
```

The template can be omitted if the new graph should contain triples identical to those in the WHERE clause pattern.

CONSTRUCT returns new RDF triples. Instead of just returning a tabular list of variable bindings, like the SELECT query, a CONSTRUCT query returns an RDF graph.

This is particularly useful for transforming, generating, filtering and sharing RDF data.

CONSTRUCT uses the template and binds variables in the WHERE part.

Later, in the SPARQL Update Language section, we'll see that we can use the CONSTRUCT query first to verify that the changes we intend to make with INSERT are as expected.



ASK query form

Does customer "Smith and Co" exists have a telephone number

```
ASK {  
  ?cust a ex:Customer .  
  ?cust rdfs:label "Smith and Co" .  
  ?cust ex:tel ?tel .  
}
```

Did the load data work? Is there any data in the graph?

```
ASK { ?s ?p ?o }
```

ASK is used mostly in conditional logic within an application or script.

ASK determines whether a query pattern matches. It returns a boolean value (true or false) if any match are found.

ASK is often used just to check data has been loaded as expected or modified as expected.

ASK mitigates the need for an App to use SELECT followed by checking whether the result set is empty or not.



DESCRIBE query form

Show all triples where :x123 is the subject

```
DESCRIBE :x123
```

Describe all triples beginning with iri's
identified by a graph pattern

```
DESCRIBE ?x {  
    ?x a ex:Customer ;  
    ex:Name []  
}
```

The DESCRIBE is used to return a description of resources that match the query pattern. It returns RDF as opposed to variable bindings in a tabular view.

It returns all RDF triples from the database where the matched resource appears as the subject.

Each vendor can implement DESCRIBE as they wish, and its behavior may be configurable. Check the docs for the specific SPARQL endpoint you are using.



NEXT:

Named Graphs

Sparql Update Language



Named Graphs – Overview

An extension to RDF allows triples to be grouped and referred to by a name (IRI). These groupings are called Named Graphs.

Named graphs are useful for many purposes:

- **Convenience:** 'Tie a bow' around a set of triples for a number of reasons, including debug, comparison of datasets and checking edit queries safely.
- **Data Provenance:** Track the origin of data
- **Contextualization:** Manage different contexts and versions, without merging.
- **Access Control:** Apply security and privacy controls to specific subsets of data.
- **Query Optimization:** Improve performance of SPARQL queries by allowing operations on specific named subsets of data.



Named Graphs – Inserting and Deleting triples

Up to now we have used only the default unnamed graph. EG in turtle...

```
:x123 a ex:Customer
```

We can read and write data to a named graph using the GRAPH keyword as follows

```
INSERT DATA { GRAPH :myGraph
```

```
    { :myString rdf:value "Hello World" } # write this triple in a graph named :myGraph  
}
```

Notice the format is now Trig which is an extension to Turtle to support named graphs

And read all triples from the named graph similarly

```
select * { GRAPH :myGraph { ?s ?p ?o } }
```

Here we treat the data as **quads**, where the fourth element specifies the named graph.



Inserting sample data into Named Graphs

Suppose we insert the following triples

```
INSERT DATA {
```

```
  GRAPH ex:Sales_2023 {
```

```
    :c01 :buys :p01, :p02 .
```

```
    :c03 :buys :p03 .
```

```
  }
```

```
  GRAPH ex:Sales_2024 {
```

```
    :c01 :buys :p01, :p03 .
```

```
    :c02 :buys :p02 , :p04 .
```

```
    :c03 :buys :p04 .
```

```
  }
```

```
}
```

Here we have created two named graphs to capture sales data linking customer to product for 2 separate years. Using named graphs allows us to keep the sales data for different years separate.

INSERT DATA is usually used only for small datasets or for testing and debug purposes.

We can query which graphs exist and query the data in the graphs.



Querying Named Graphs

We can query just one graph by explicitly specifying it.

```
select * { GRAPH ex:Sales_2023 { ?s ?p ?o }}
```

Or query both graphs explicitly using UNION

ie $g1 \cup g2$

```
select * {  
  { GRAPH ex:Sales_2023 { ?s ?p ?o } }  
  UNION  
  { GRAPH ex:Sales_2024 { ?s ?p ?o } }  
}
```

Or we can grab the data from all named graphs, using a variable eg ?g, without specifically naming the graphs

```
select * { GRAPH ?g { ?s ?p ?o } }
```

Querying a named graph is similar to querying the default nonamed graph.

The primary difference is that you must specify the graph to query, either by explicitly naming the graph or using a variable to reference it.

If you omit the GRAPH keyword the query queries the default graph. IE The default graph is accessed implicitly by omitting the GRAPH keyword.



Querying Named Graphs

If you need to, for clarity, specify the nonnamed graph most Sparql endpoints will allow it. EG

```
SELECT * {  
  GRAPH <tag:stardog:api:context:default> {  
    ?s ?p ?o  
  }  
}
```

To access the default graph you just omit the GRAPH keyword but most Sparql endpoints will allow you to explicitly specify the nonnamed graph.



Named Graphs $g_1 \cap g_2$ and g_1/g_2

Finding triples that are in both graphs

ie $g_1 \cap g_2$

```
select * {  
  graph ex:Sales_2023 { ?s ?p ?o }  
  graph ex:Sales_2024 { ?s ?p ?o }  
}
```

Or ask what's in g_1 not in g_2 using MINUS

ie g_1/g_2

```
select * {  
  { graph ex:Sales_2023 { ?s ?p ?o } }  
  MINUS  
  { graph ex:Sales_2024 { ?s ?p ?o } }  
}
```

Comparing datasets is straightforward.

The first query finds the intersection between the two graphs. This is because the variables ?s, ?p, and ?o must have the same value in both graphs for a match to occur.

The second query find triples in 2023 but not 2024. It matches all triples in ex:Sales_2023 with the pattern ?s ?p ?o. Then, it excludes any of those triples that also appear in ex:Sales_2024.



Named Graphs – Datasets

So far we have used QUAD notation to access named graphs.

- **Quad Notation** (using **GRAPH**) makes it clear that the query is dealing with a specific named graph within a potentially larger dataset, explicitly adding the fourth element (or variable) to the triple.

There is an alternative powerful notation that allows more flexibility. You can specify the dataset for the query is to comprise of zero or more named graphs. If no named graphs are specified, only the default nonamed graph is queried.

- **Dataset Notation** (using **FROM**) indicates that the entire query is scoped to a specific dataset, which may be a single named graph or multiple named graphs. It sets the context for the whole query but does not explicitly add the graph name to each triple pattern in the **WHERE** clause.



Dataset Notation – FROM keyword

Before named graphs were introduced, there was only a single graph per database. This graph served as the context for all queries and updates. It did not need a name.

When named graphs were introduced, for more flexible data management, Sparql provided a mechanism to scope queries to specific named graphs. We can change the 'default graph' to be, for example, the named graphs :g1 and :g2 using FROM

```
SELECT *  
FROM :g1  
FROM :g2  
WHERE { ?s ?p ?o }
```

Here the triples from both graphs :g1 and :g2 are combined and treated as if they were part of a single graph, effectively creating a temporary union of the specified graphs. This combined graph acts as the default graph for the query, meaning that we do not need (and cannot use) the GRAPH keyword to access data from :g1 and :g2 explicitly within this query.



DATASET notation – FROM NAMED

First place a triple in the default graph

```
insert data { :foo :boo "bar" }
```

Access only the nonnamed graph

```
Select * {?s ?p ?o}
```

Comparing datasets is straightforward.

The first query finds the intersection between the two graphs. This is because the variables ?s, ?p, and ?o must have the same value in both graphs for a match to occur.

The second query find triples in 2023 but not 2024. It matches all triples in ex:Sales_2023 with the pattern ?s ?p ?o. Then, it excludes any of those triples that also appear in ex:Sales_2024.



Sparql Update Language – quick intro

Sparql Update Language allows us to insert, edit and delete RDF data.

Insert Data adds triples to a graph. No variables allowed! Triples are specified explicitly.

```
INSERT DATA { :myString rdf:value "Hello World" .  
               :pluto a :Planet ; rdfs:label "Pluto" }
```

Delete Data removes triples from a graph. Triples are specified explicitly.

```
DELETE DATA { :pluto a :Planet ; rdfs:label "Pluto" }
```



Review

Review

- Key terminology including dataset and triples
- Basic Graph Patterns (BGP)
- Query language examples
- Named Graphs
- SPARQL Update Language introduction



Questions

Support Resources

- Self Help:
 - Community Portal <https://community.stardog.com>
 - Stardog Academy <https://academy.stardog.com>
 - Documentation <https://docs.stardog.com>
- File a Ticket:
 - Via Email: support@stardog.com
 - Via Help Desk: <https://support.stardog.com>



Thank You!



